

Nucleotide Sequence Analysis Toolkit Using Python and Biopython: Composition, ORFs, Restriction Sites, and Translation

Introduction: This is a python script; it performs nucleotide sequence analysis using Biopython. It accepts FASTA format as input and identifies whether the input sequence is a valid nucleotide sequence and tells us whether it is a DNA or RNA sequence. Then the script calculates length, nucleotide residues composition, purine and pyrimidine percentages, generates complement and reverse complement, transcription or back transcription (if the input is an RNA sequence) and translates the sequence into a protein sequence. It also scans for start and stop codons, restriction sites and gives us its position if there are any and visualizes nucleotide distribution with a bar graph.

Methods: Python version 3.13.9 and biopython version 1.86 is used to program the script. Biopython, matplotlib are installed. Line breaks and spaces were removed to generate continuous sequence string. The program classifies the sequence as DNA or RNA based on the presence of T or U. Total nucleotide count/length was obtained using len() function. A, G, C, T/U were calculated using .count() function. Purine and pyrimidine percentages were analysed relative to the total sequence length. Seq methods from biopython library was used to generate complement, reverse complement, transcription, reverse transcription, translation. Start and stop codons, restriction enzymes were scanned and their positions were obtained. Library matplotlib was used to plot nucleotide composition bar chart.

Following codes were taken as references

1) Bioinformage. (2025). *Dna_rna_protein_seq.py* [Source code].

GitHub. https://github.com/Bioinformage/Biopython-scripts-collection/blob/main/DNA_RNA_and_Protein_Sequence_Analysis/Dna_rna_protein_seq.py

2) mike-mike-mike-mike-mike. (2025). *DNA Codon Methods.py*

[Source code]. GitHub. https://github.com/mike-mike-mike-mike-mike/DNA_Python_Tools/blob/main/DNA%20Codon%20Methods.py

Screen shots of the script is shared below:

```
1  from Bio.Seq import Seq
2  import os
3  import sys
4
5  #paste your FASTA sequence inside the triple quotes without any header
6  your_fasta = """ """
7
8  #FASTA might have line breakes, spaces
9  clean = your_fasta.replace(_old: "\n", _new: "").replace(_old: " ", _new: "")
10  fasta = Seq(clean)
11
12  #identify whether the sequence is a valid DNA or RNA sequence, if it is not a valid sequence exit
13  if "U" in fasta and "T" in fasta:
14      sys.exit("The sequence is not valid")
15  # identify whether the sequence is DNA or RNA
16  elif "T" in fasta:
17      print("It is a DNA sequence")
18  elif "U" in fasta :
19      print("It is a RNA sequence")
20
21  length = len(fasta) #length of the DNA
22  print('Nucleotide count:',length)
23
24  #check G,A,C,T or U content in sequence
25  G = fasta.count('G')
26  print("Number of Guanine residues in sequence:", G)
27  A = fasta.count('A')
28  print("Number of Adenine residues in sequence:", A)
29  C = fasta.count('C')
30
```

```
31  print("Number of Cytosine residues in sequence:", C)
32  if "U" in fasta:
33      U = fasta.count("U")
34      print("Number of Uracil residues in sequence:", U)
35  if "T" in fasta:
36      T = fasta.count("T")
37      print("Number of Thymine residues in sequence:", T)
38
39  purines = fasta.count("A") + fasta.count("G") #purines content in percentage
40  purine_percentage = purines / length * 100
41  print("Purines (Adenine + Guanine) content %:",purine_percentage)
42  pyrimidines = fasta.count("C") #pyrimidines content in percentage
43  if "U" in fasta:
44      pyrimidines += fasta.count("U")
45  if "T" in fasta:
46      pyrimidines += fasta.count("T")
47  pyrimidines_percentage = pyrimidines / length * 100
48  print("Pyrimidines (Cytosine + (Uracil or Thymine) content %:",pyrimidines_percentage)
49
50  print("Complement of the sequence:", fasta.complement()) #complement of sequence
51  print("Reverse complement of the sequence:", fasta.reverse_complement()) #reverse complement of sequence
52  if "U" in fasta: #transcription of sequence
53      RNA_transcribe = fasta.transcribe()
54      print("RNA to DNA back transcription:", RNA_transcribe)
55  if "T" in fasta:
56      print("DNA to RNA transcription:", fasta.transcribe())
57  if "T" in fasta: #translation of nucleotide sequence into protein sequence
58      print("DNA to protein translation:", fasta.translate())
59  if "U" in fasta:
60      RNA_trans = RNA_transcribe.translate()
61      print("RNA to protein translation:", RNA_trans)
62
```

```

61     print("RNA to protein translation:", RNA_trans)
62
63     #to check for start and stop codons in sequence
64     start_position = []
65     stop_position = []
66     if "T" in fasta:
67         start_codon = "ATG"
68         stop_codon = ["TAA", "TAG", "TGA"]
69         for i in range(0, len(fasta) - 2, 3):
70             codon = str(fasta[i:i + 3])
71             if codon == start_codon:
72                 start_position.append(i)
73             if codon in stop_codon:
74                 stop_position.append(i)
75     if "U" in fasta:
76         start_codon = "AUG"
77         stop_codon = ["UAA", "UAG", "UGA"]
78         for i in range(0, len(fasta) - 2, 3):
79             codon = str(fasta[i:i + 3])
80             if codon == start_codon:
81                 start_position.append(i)
82             if codon in stop_codon:
83                 stop_position.append(i)
84     print("start codon positions:",start_position)
85     print("stop codon positions:",stop_position)
86
87     #check for restriction sites in sequence
88     seq = str(fasta)
89     restriction_sites = {
90         "EcoRI": "GAATTC",
91         "HindIII": "AAGCTT",
92         "BamHI": "GGATCC",

```

```

93         "NotI": "GCGGCCGC",
94         "PstI": "CTGCAG",
95         "SphI": "GCATGC"
96     }
97     for name, site in restriction_sites.items():
98         start = 0
99         while True:
100             pos = seq.find(site, start)
101             if pos == -1:
102                 break
103             print(f"{name} ({site}) found at position {pos}")
104             start = pos + 1
105
106     #plot Nucleotide composition bar graph
107     import matplotlib.pyplot as plt
108     from collections import Counter
109     seq = str(fasta)
110     counts = Counter(seq)
111     bases = sorted(counts.keys())
112     values = [counts[b] for b in bases]
113     plt.figure(figsize=(6,3))
114     plt.bar(bases, values)
115     plt.title("Nucleotide composition")
116     plt.xlabel("Nucleotide Bases")
117     plt.ylabel("Count")
118     plt.tight_layout()
119     plt.show()

```

Results: A part of nucleotide coding sequence was extracted from monkeypox virus, clade IIa in FASTA format. Following results were achieved.

```

1  from Bio.Seq import Seq
2  import os
3  import sys
4
5  #paste your FASTA sequence inside the triple quotes without any header
6  your_fasta = """ATGGAGTCGGTAGATTTTCATGGCTGTCGATGAGCAGTTTCACGACGACCTCGATCTTTGGTCATTATCTT
7  TGGTAGATGATTATAAAAAACATGGATTAGGTGTTGACTGTTATGTTCTAGAACCAGTTGTTGACAGGAA
8  AATATTTGATAGATTTCTCCTTGAACCAATTTGTGATCCTGTAGATGTTCTGTATGATTATTTAGGATT
9  CATAGAGATAATATTGATCAGTATATAGTAGATAGACTGTTTGCATATATTACATATAAAGATATTATAT
10 CTGCAATAGTGTCAGGAATATATGGAAGATATTTTCTCTATAATTATTAAGAATTGTAATTCGTGCA
11 AGATCTCTTACTTTACTATCTATCTAATGCATATGTAGAAATAGACATTGTTGATCTTATGGTAGATCAT
12 GGGGCTGTAATATATAAAATAGAATGCTTGAATGCCTATTTTAGGGGAATATGTAAGGAAAGTAAAGT
13 TTGTTGAGTTTATTTGAATTGTGGTATCCAGATGAAATGATGTTAAATTAGATCTATATAAAATAAT
14 TCAGTATACTAGGGGATTCCTTGTAGATGAACCCACAGTATTAGAAATTTATAAGCTTTGTATCCCATAT
15 ATTGAAGATATCAATCACTAGATGCTGGTGGAGGACCTTGCTTTATCGCGCTATCTATGCAGGTTATA
16 TAGATTTAGTATCATTGCTATTAGAAAATGGAGCAATGTCAACGCAGTAATGAGTAATGGATATACATG
17 TCTTGACGTGGCCGTGGATAGGGGATCTGTCATCGCCCGTAGGGAAGCACATCTTAAATATTAGAAATA
18 TTGCTTAGAGAACCATTTGCTATTGACTGTATAAAATTAGCTATACTTAATAATACAATTGAAAACCATG
19 ATGTGATAAAGCTCTGTATCAAGTATTTTATGATGGTAGATTATTCACCTTTGTAATGTGTATGCATCATC
20 ACTCTTTTATTATATAATTGATTGTAAACAAGAATTGGAGTACATTAGGCAGATGAAAATTCATAATACA
21 ACCATGTATGAGTTAATCTATAATAGAGACAAAACAAGCATGCTTCCCATATTCTACATAGGTATTCTA
22 AACATCCAGTTTTGACACAGTGTATCACTAAAGGATTCAAGATTTACACAGAAGTAACCGAGCAGGTCAC
23 TAAAGCTCTAAACAGACGTGCTCTAATAGATGAGATAATAACAATGTATCACTGATGACAATCTCCTA
24 TCAAACTTCCATTAGAAATTAGGGATCTAATCGTTTCACAAGCTGTCATATAG|"""
25
26 #FASTA might have line breaks, spaces
27 clean = your_fasta.replace(_old: "\n", _new: "").replace(_old: " ", _new: "")
28 fasta = Seq(clean)

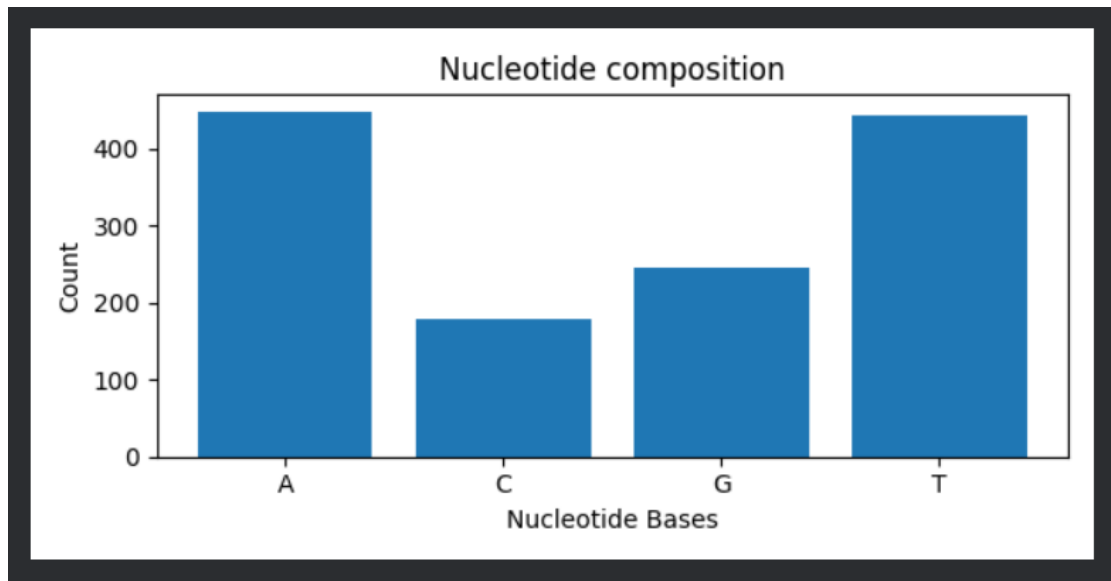
```

```

Run learn2 x
C:\Users\sachi\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\sachi\PyCharmMiscProject\.venv\learn2.py
It is a DNA sequence
Nucleotide count: 1314
Number of Guanine residues in sequence: 245
Number of Adenine residues in sequence: 448
Number of Cytosine residues in sequence: 178
Number of Thymine residues in sequence: 443
Purines (Adenine + Guanine) content %: 52.73972602739726
Pyrimidines (Cytosine + Uracil or Thymine) content %: 47.26027397260274
Complement of the sequence: TACCTCAGCATCTAAAGTACCGACAGTACTGCTAAAGTGCTGCTGGAGCTAGAAACCGAATAGAAACCATCTACTAATATTTTGTACCTAATCCAACTGACAATACAGATCTTGGTCAACAAGTGTCTTTTATAAG
Reverse complement of the sequence: CTATATGACAGCTTGTGAACGATTAGATCCCTAATTTCTAATGGAAGTTTGTATGAGAGATTGTCATCAGTTGATACATGTTTATTATCTCATCTATTAGAGCACGCTGTTTAGAGCTTTAGTGACCTGCTCGGTTA
DNA to RNA transcription: AUGGAGUCGUGAGAUUUCAGGUCUGUCAGUAGCAGUUUCAGCAGCAGCUCGUAUCUUUGUAGAUUAUAAAAACUAGGAUAGGUGUAGCUGUUAUGUUCUAGAACCAAGUUGUAGCAGGAAAAUUAUUGAL
DNA to protein translation: MESVDFMAVDEQFHDLDLWLSLVDYKXHLGVDCYVLEPVVDRKIFDRFLLEPICDPVDVLYDFRIHRONIDQYIVORLFAYITYKDIISALVSKNYMEDIFSIIKNCMSVDQLLLYLSNAYVEIDIVLHVHGAIVYKIE
start codon positions: [0, 18, 303, 408, 750, 939, 942, 1032, 1053]
stop codon positions: [1311]
HindIII (AAGCTT) found at position 612
SphI (GCATGC) found at position 1088

Process finished with exit code 0

```



Summary: The script processed the input FASTA sequence by removing line breaks, blank spaces. And confirming that it is a valid DNA sequence. Nucleotide length/count, individual nucleotides count, purine and pyrimidine percentage was correctly calculated. Reverse complement, complement, transcription, translation was generated. Location of start and stop codons, restriction sites were printed. Nucleotide composition was visualized using a bar graph, illustrating the distribution of bases within the sequence.